

Department Of Computer Science
AI-02

OOPs Project Report

Hotel Management System Using C++



Group Members:

- Sami-ul-Haq - 250901010
- Shah Hassan Nawab – 250901012
- Saad Khan – 250901014
- Muhammad Sahil Khan – 250901032

Submitted To: Ms. Fizza

Institute of Space Technology Islamabad

Contents

Project title:-	2
Hotel Management System using C++ (Object Oriented Programming)	2
1. ABSTRACT	2
2. INTRODUCTION	2
3. OBJECTIVES	2
4. PROBLEM STATEMENT.....	3
5. FEATURES OF SYSTEM.....	3
6. OOP CONCEPTS USED.....	3
1. Encapsulation	3
2. Inheritance	3
3. Polymorphism	3
4. Abstraction	3
5. Dynamic Binding.....	4
6. Constructors & Destructors	4
7. SYSTEM MODULES.....	4
1. Room Management Module	4
2. Customer Management Module.....	4
3. Booking Module	4
4. Billing Module	4
5. Checkout Module	4
6. File Handling Module.....	4
7. CSV Export Module	4
8. UML DIAGRAM.....	5
9. FLOWCHART	5
10. CODE SECTION.....	6
11. SCREENSHOTS.....	12
12. ADVANTAGES	16
13. LIMITATIONS	16
14. FUTURE ENHANCEMENTS.....	16
15. CONCLUSION.....	16

Project title:-

Hotel Management System using C++ (Object Oriented Programming)

1. ABSTRACT

The Hotel Management System is a console-based application developed in C++ that automates hotel operations such as room booking, customer registration, billing, checkout, and data management.

The system replaces manual record-keeping with a computerized solution that improves efficiency, reduces errors, and enhances data accuracy. It uses Object-Oriented Programming (OOP) concepts including inheritance, polymorphism, abstraction, and encapsulation.

The system also supports file handling and CSV export to store data permanently and open it in Excel for reporting purposes.

2. INTRODUCTION

Hotels require efficient management of rooms, customers, and billing systems. Manual systems are slow and error-prone. To solve this, a C++ based Hotel Management System is developed.

This system helps:

- Manage room availability
- Store customer records
- Automate booking process
- Generate bills
- Perform checkout operations
- Store data permanently

3. OBJECTIVES

The main objectives of this project are:

- **To develop a hotel management system using C++**
- **To apply Object-Oriented Programming concepts**
- **To automate hotel operations**
- **To manage customer and room records efficiently**
- **To generate accurate billing system**
- **To implement file handling for permanent storage**
- **To export data into CSV (Excel format)**

4. PROBLEM STATEMENT

Manual hotel management has several issues:

- **Data duplication**
- **Human errors in billing**
- **Difficult record tracking**
- **Slow processing**
- **No proper data storage system**

This project solves these problems using a structured OOP-based system.

5. FEATURES OF SYSTEM

- ✓ Add Rooms
- ✓ Display Available Rooms
- ✓ Book Rooms
- ✓ Store Customer Data
- ✓ Search Customer/Room Data
- ✓ Generate Bills
- ✓ Checkout Customers
- ✓ File Handling (.txt storage)
- ✓ Export Data to CSV (Excel support)

6. OOP CONCEPTS USED

1. Encapsulation

Data is hidden inside classes using private variables.

2. Inheritance

- **Customer inherits Person class**
- **StandardRoom & DeluxeRoom inherit Room class**

3. Polymorphism

- **Virtual functions used:**
 - **display()**
 - **bill()**
 - **getType()**

4. Abstraction

Abstract classes:

- **Person**
- **Room**

5. Dynamic Binding

Achieved using virtual functions

6. Constructors & Destructors

Used for initialization and cleanup

7. SYSTEM MODULES

1. Room Management Module

- **Add rooms**
- **Display rooms**
- **Check availability**

2. Customer Management Module

- **Add customer data**
- **Store records**

3. Booking Module

- **Assign rooms to customers**
- **Check availability**

4. Billing Module

- **Calculate total cost based on days**

5. Checkout Module

- **Free occupied rooms**

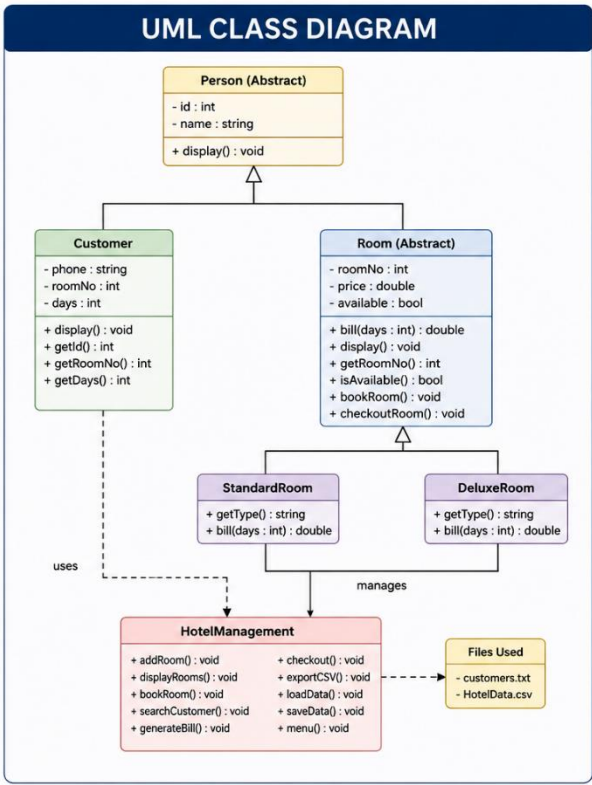
6. File Handling Module

- **Store data in text file**
- **Load previous records**

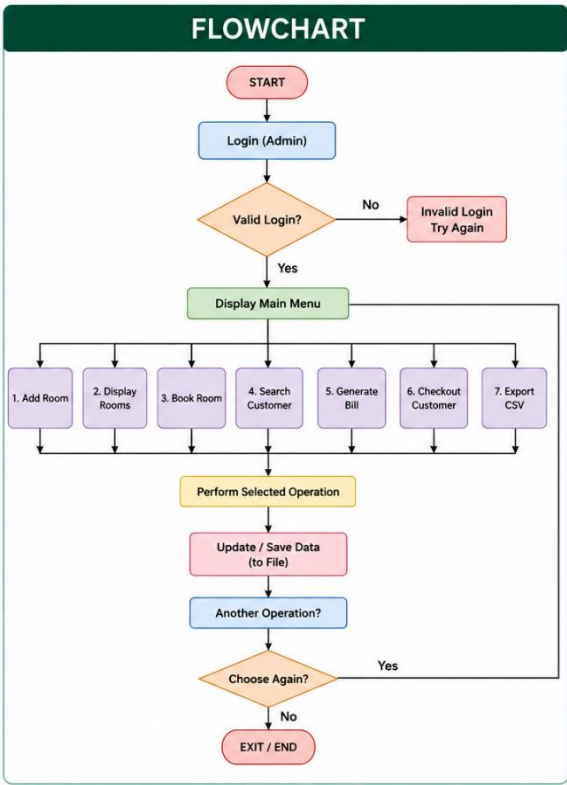
7. CSV Export Module

- **Export data to Excel format**

8. UML DIAGRAM



9. FLOWCHART



10. CODE SECTION

```
#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>
using namespace std;

/* =====
   ABSTRACT CLASS (OOP)
   ===== */
class Person {
protected:
    int id;
    string name;

public:
    Person(int i = 0, string n = "") {
        id = i;
        name = n;
    }

    virtual void display() = 0; // Abstraction
};

/* =====
   CUSTOMER CLASS (Inheritance)
   ===== */
class Customer : public Person {
    string phone;
    int roomNo;
    int days;

public:
    Customer(int i = 0, string n = "", string p = "", int r = 0, int d = 0)
        : Person(i, n) {
        phone = p;
        roomNo = r;
        days = d;
    }

    int getId() { return id; }
    int getRoomNo() { return roomNo; }
    int getDays() { return days; }
    string getName() { return name; }
    string getPhone() { return phone; }

    void setRoom(int r, int d) {
        roomNo = r;
        days = d;
    }

    void display() override {
```

```

        cout << "\nID: " << id
            << "\nName: " << name
            << "\nPhone: " << phone
            << "\nRoom: " << roomNo
            << "\nDays: " << days << endl;
    }
};

/* =====
ROOM CLASS (Inheritance + Polymorphism)
===== */
class Room {
protected:
    int roomNo;
    double price;
    bool available;

public:
    Room(int r, double p) {
        roomNo = r;
        price = p;
        available = true;
    }

    virtual string getType() = 0;

    int getRoomNo() { return roomNo; }
    bool isAvailable() { return available; }

    void bookRoom() { available = false; }
    void checkoutRoom() { available = true; }

    virtual double bill(int days) {
        return price * days;
    }

    virtual void display() {
        cout << "Room: " << roomNo
            << " | Price: " << price
            << " | Status: " << (available ? "Available" : "Booked")
            << endl;
    }

    virtual ~Room() {}
};

/* =====
STANDARD ROOM
===== */
class StandardRoom : public Room {
public:
    StandardRoom(int r) : Room(r, 3000) {}

    string getType() override {
        return "Standard";
    }
};

```



```

    }
};

/* =====
   DELUXE ROOM
   ===== */
class DeluxeRoom : public Room {
public:
    DeluxeRoom(int r) : Room(r, 6000) {}

    string getType() override {
        return "Deluxe";
    }
};

/* =====
   HOTEL MANAGEMENT SYSTEM
   ===== */
class HotelManagement {
private:
    vector<Customer> customers;
    vector<Room*> rooms;

public:
    HotelManagement() {
        loadData();

        // default rooms
        if (rooms.empty()) {
            rooms.push_back(new StandardRoom(101));
            rooms.push_back(new StandardRoom(102));
            rooms.push_back(new DeluxeRoom(201));
            rooms.push_back(new DeluxeRoom(202));
        }
    }

    /* ===== FILE LOAD ===== */
    void loadData() {
        ifstream file("customers.txt");
        int id, room, days;
        string name, phone;

        while (file >> id >> name >> phone >> room >> days) {
            customers.push_back(Customer(id, name, phone, room, days));
        }
        file.close();
    }

    /* ===== SAVE FILE ===== */
    void saveData() {
        ofstream file("customers.txt");

        for (auto &c : customers) {
            file << c.getId() << " "
                << c.getName() << " "

```

```

        << c.getPhone() << " "
        << c.getRoomNo() << " "
        << c.getDays() << endl;
    }
    file.close();
}

/* ===== ADD ROOM ===== */
void addRoom() {
    int type, no;

    cout << "\n1. Standard\n2. Deluxe\nEnter type: ";
    cin >> type;

    cout << "Enter Room Number: ";
    cin >> no;

    if (type == 1)
        rooms.push_back(new StandardRoom(no));
    else
        rooms.push_back(new DeluxeRoom(no));

    cout << "Room Added Successfully!\n";
}

/* ===== DISPLAY ROOMS ===== */
void displayRooms() {
    cout << "\n===== ROOMS =====\n";
    for (auto r : rooms) {
        r->display();
    }
}

/* ===== SEARCH CUSTOMER ===== */
void searchCustomer() {
    int id;
    cout << "Enter Customer ID: ";
    cin >> id;

    for (auto &c : customers) {
        if (c.getId() == id) {
            c.display();
            return;
        }
    }
    cout << "Customer Not Found!\n";
}

/* ===== BOOK ROOM ===== */
void bookRoom() {
    int id, roomNo, days;
    string name, phone;

    cout << "Enter Customer ID: ";
    cin >> id;

```

```

        cout << "Enter Name: ";
        cin >> name;

        cout << "Enter Phone: ";
        cin >> phone;

        cout << "Enter Room No: ";
        cin >> roomNo;

        cout << "Enter Days: ";
        cin >> days;

        for (auto r : rooms) {
            if (r->getRoomNo() == roomNo && r->isAvailable()) {

                r->bookRoom();

                customers.push_back(Customer(id, name, phone, roomNo, days));

                cout << "\nRoom Booked Successfully!\n";
                saveData();
                return;
            }
        }

        cout << "Room Not Available!\n";
    }

    /* ===== GENERATE BILL ===== */
    void generateBill() {
        int id;
        cout << "Enter Customer ID: ";
        cin >> id;

        for (auto &c : customers) {
            if (c.getId() == id) {

                for (auto r : rooms) {
                    if (r->getRoomNo() == c.getRoomNo()) {

                        double total = r->bill(c.getDays());

                        cout << "\n==== BILL =====";
                        cout << "\nName: " << c.getName();
                        cout << "\nRoom: " << c.getRoomNo();
                        cout << "\nDays: " << c.getDays();
                        cout << "\nTotal Bill: " << total << endl;
                        return;
                    }
                }
            }
        }

        cout << "Customer Not Found!\n";
    }

```

```

}

/* ===== CHECKOUT ===== */
void checkout() {
    int id;
    cout << "Enter Customer ID: ";
    cin >> id;

    for (int i = 0; i < customers.size(); i++) {
        if (customers[i].getId() == id) {

            for (auto r : rooms) {
                if (r->getRoomNo() == customers[i].getRoomNo()) {
                    r->checkoutRoom();
                }
            }

            customers.erase(customers.begin() + i);
            cout << "Checkout Successful!\n";

            saveData();
            return;
        }
    }

    cout << "Customer Not Found!\n";
}

/* ===== EXPORT CSV ===== */
void exportCSV() {
    ofstream file("HotelData.csv");

    file << "ID,Name,Phone,Room,Days\n";

    for (auto &c : customers) {
        file << c.getId() << ","
              << c.getName() << ","
              << c.getPhone() << ","
              << c.getRoomNo() << ","
              << c.getDays() << "\n";
    }

    file.close();

    cout << "CSV Exported Successfully!\n";
}

/* ===== MENU ===== */
void menu() {
    int choice;

    do {
        cout << "\n\n==== HOTEL MANAGEMENT SYSTEM =====\n";
        cout << "\n1. Add Room";
        cout << "\n2. Display Rooms";
    } while (choice != 0);
}

```

```

        cout << "\n3. Book Room";
        cout << "\n4. Search Customer";
        cout << "\n5. Generate Bill";
        cout << "\n6. Checkout Customer";
        cout << "\n7. Export CSV";
        cout << "\n8. Exit";

        cout << "\nEnter Choice: ";
        cin >> choice;

        switch (choice) {
            case 1: addRoom(); break;
            case 2: displayRooms(); break;
            case 3: bookRoom(); break;
            case 4: searchCustomer(); break;
            case 5: generateBill(); break;
            case 6: checkout(); break;
            case 7: exportCSV(); break;
            case 8: saveData(); break;
            default: cout << "Invalid Choice!\n";
        }

    } while (choice != 8);
}

~HotelManagement() {
    for (auto r : rooms)
        delete r;
}
};

/* ===== MAIN ===== */
int main() {
    HotelManagement h;
    h.menu();
    return 0;
}

```

11. SCREENSHOTS

Screenshot 1: Main Menu

```

PS C:\Users\HP> cd "d:\BS AI\2nd Semester\OOPS Lab\Project"

===== HOTEL MANAGEMENT SYSTEM =====
1. Add Room
2. Display Rooms
3. Book Room
4. Search Customer
5. Generate Bill
6. Checkout Customer
7. Export CSV
8. Exit
Enter Choice: █

```

Screenshot 2: Add Room

```
===== HOTEL MANAGEMENT SYSTEM =====
1. Add Room
2. Display Rooms
3. Book Room
4. Search Customer
5. Generate Bill
6. Checkout Customer
7. Export CSV
8. Exit
Enter Choice: 1

1. Standard
2. Deluxe
Enter type: 1
Enter Room Number: 500
Room Added Successfully!
```

Screenshot 3: Display Rooms

```
===== HOTEL MANAGEMENT SYSTEM =====
1. Add Room
2. Display Rooms
3. Book Room
4. Search Customer
5. Generate Bill
6. Checkout Customer
7. Export CSV
8. Exit
Enter Choice: 2

===== ROOMS =====
Room: 101 | Price: 3000 | Status: Booked
Room: 102 | Price: 3000 | Status: Available
Room: 201 | Price: 6000 | Status: Available
Room: 202 | Price: 6000 | Status: Available
Room: 500 | Price: 3000 | Status: Available
```

Screenshot 4: Book Room

```
===== HOTEL MANAGEMENT SYSTEM =====
1. Add Room
2. Display Rooms
3. Book Room
4. Search Customer
5. Generate Bill
6. Checkout Customer
7. Export CSV
8. Exit
Enter Choice: 3
Enter Customer ID: 111
Enter Name: Sahil
Enter Phone: 03439995048
Enter Room No: 101
Enter Days: 4

Room Booked Successfully!
```

Screenshot 5: Search Customers

```
===== HOTEL MANAGEMENT SYSTEM =====
1. Add Room
2. Display Rooms
3. Book Room
4. Search Customer
5. Generate Bill
6. Checkout Customer
7. Export CSV
8. Exit
Enter Choice: 4
Enter Customer ID: 111

ID: 111
Name: Sahil
Phone: 03439995048
Room: 101
Days: 4
```

Screenshot 6: Billing Output

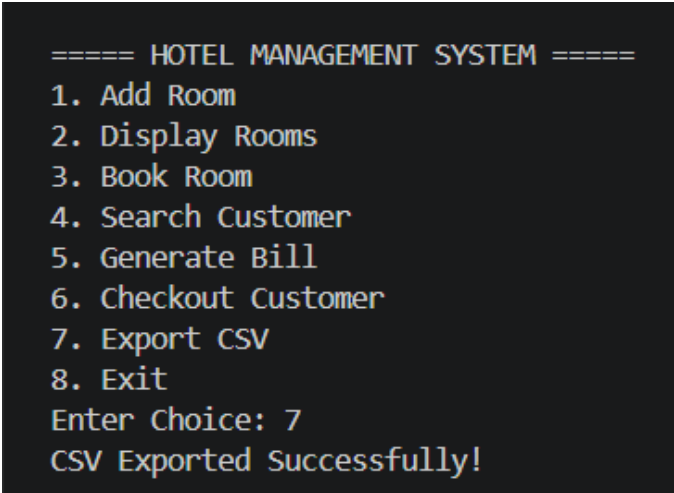
```
===== HOTEL MANAGEMENT SYSTEM =====
1. Add Room
2. Display Rooms
3. Book Room
4. Search Customer
5. Generate Bill
6. Checkout Customer
7. Export CSV
8. Exit
Enter Choice: 5
Enter Customer ID: 111

===== BILL =====
Name: Sahil
Room: 101
Days: 4
Total Bill: 12000
```

Screenshot 7: Checkout

```
===== HOTEL MANAGEMENT SYSTEM =====
1. Add Room
2. Display Rooms
3. Book Room
4. Search Customer
5. Generate Bill
6. Checkout Customer
7. Export CSV
8. Exit
Enter Choice: 6
Enter Customer ID: 111
Checkout Successful!
```

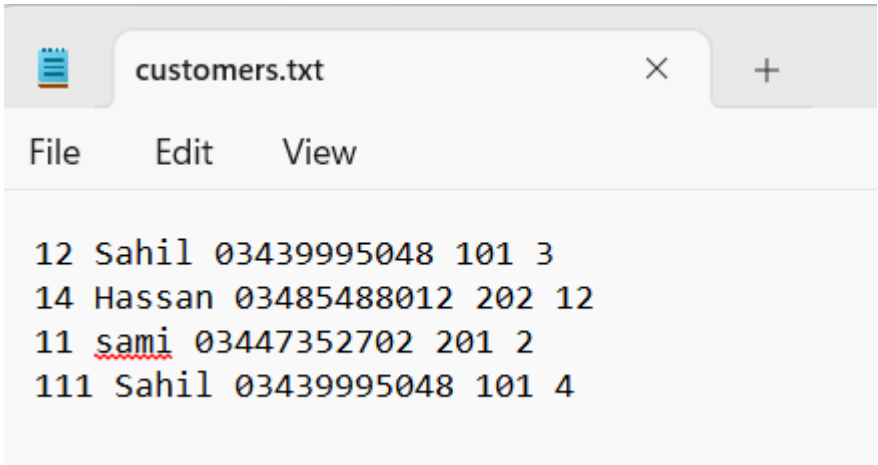
Screenshot 8: CSV Export



Screenshot 9: Records Save in Excel

A1	:				ID					
	A	B	C	D	E	F	G			
1	ID	Name	Phone	Room	Days					
2	12	Sahil	3439995048	101	3					
3	14	Hassan	3485488012	202	12					
4	11	sami	3447352702	201	2					
5	111	Sahil	3439995048	101	4					
6										
7										
8										
-										

Screenshot 9: Records Save in txt format



12. ADVANTAGES

- ✓ Easy to use
- ✓ Fast processing
- ✓ Reduces human errors
- ✓ Data stored permanently
- ✓ Excel support (CSV)
- ✓ Modular structure

13. LIMITATIONS

- **Console-based interface only**
- **No GUI**
- **Limited security system**
- **No online booking**

14. FUTURE ENHANCEMENTS

- **GUI-based system (Qt / Windows App)**
- **Online booking system**
- **Database integration (MySQL)**
- **Payment gateway integration**
- **Multi-user system**

15. CONCLUSION

The Hotel Management System successfully automates hotel operations using C++ and Object-Oriented Programming. It improves efficiency, reduces manual work, and provides a structured approach to managing hotel data.